

Tribhuvan University

Faculty of Humanities and Social Sciences

CanteenMate

(Canteen Management System)

Submitted to

Department of Computer Application

Everest Innovative College

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by

Dibisha Haleyo Joshi

(Reg. No.: 6-2-713-5-2021)

2025 A.D.

Under the supervision of

Mr. Vikram Yadav



Tribhuvan University

Faculty of Humanities and Social Sciences

Everest Innovative College

Soalteemode, Kalimati

Bachelor in Computer Applications (BCA)

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Dibisha Haleyo Joshi** entitled **"CanteenMate"** in the partial fulfillment of requirement for the degree of Bachelor in Computer Application is recommended for that final evaluation.

_____

Vikram Yadav

Project Supervisor

BCA Department

Everest Innovative College



Tribhuvan University

Faculty of Humanities and Social Sciences

Everest Innovative College

Soalteemode, Kalimati

LETTER OF APPROVAL

This is to certify that this project prepared by **Dibisha Haleyo Joshi** entitled **"CanteenMate"** in the partial fulfillment of requirement for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Vikram Yadav	Kishor Kafle
Supervisor	Program Coordinator
BCA Department	Everest Innovative College
Everest Innovative College	

External Examiner	Internal Examiner

ABSTRACT

CanteenMate is an easy to use canteen app designed to make canteen food ordering simple for schools and offices. It uses modern web tools to let customers order food, pay online, and see the order status, and let admin to manage orders status and print invoice, making the process fast and smooth. The app keeps things straightforward by focusing on ordering and payments. It includes a collaborative filtering algorithm like KNN, fallback that suggests menu items based on what similar customers have orders, making recommendations more personal. All data is safely stored in an SQLite database, which serves as the app's main storage. This project is built using HTM, CSS, JavaScript, Bootstrap, and Flask (Python) using Agile Methodology. CanteenMate helps canteens run efficiently, saves time, and meets the needs of today's digital world.

Keywords: canteen app, easy ordering, collaborative filtering, KNN, fallback, food suggestions, user-friendly, Agile methodology, digital solution

ACKNOWLEDGEMENT

I would like to express my special thanks of gratitude to my supervisor, **Mr. Vikram Yadav**, for providing me with the golden opportunity to work on this wonderful project on the topic of CanteenMate. This project allowed me to do a lot of research and learn about many new tools and technologies.

I would like to express my special thanks of gratitude to our Campus Chief **Mr. Dipak Kumar Jha** who gave me permission for doing this Project.

I would like to express my special gratitude and thanks to our BCA Program Coordinator **Mr. Kishor Kafle** for his support and help for our personnel development and mainly for the completion of this project.

I am highly indebted to Everest Innovative College for their guidance and constant supervision as well as for providing necessary information regarding the project and support in the completion.

I would also like to express my gratitude towards library and member of Everest Innovative College for their kind co-operation and encouragement which help us in completion of this project.

I would also like to thank my parents and friends who helped me a lot in finalizing this project within the limited time frame.

In the end, I would like to thank Tribhuvan University for offering this opportunity via the course of Computer application, which helped me understand project ethics at this early stage and helped me to evaluate my knowledge and expand it a little more.

Yours sincerely,

Dibisha Haleyo Joshi

Contents

LIST OF ABBREVIATIONS vii

LIST OF FIGURES viii

LIST OF TABLES ix

CHAPTER 1: INTRODUCTION 1

1.1 Introduction 1

1.2 Problem Statement 2

1.3 Objective 2

1.4 Scope and Limitations 2

1.4.1 Scope 2

1.4.2 Limitations 3

1.5 Development Methodology 3

1.6 Report Organization 4

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW 5

2.1 Background Study 5

2.2 Literature Review 5

CHAPTER: 3 SYSTEM ANALYSIS AND DESIGN 7

3.1 System Analysis 7

3.1.1 Requirement Analysis 7

3.1.2 Feasibility Analysis 9

3.1.3 Data Modeling 10

3.1.4 Process Modeling: DFD 12

3.2 System Design 14

3.2.1 Architectural Design 14

3.2.2 Database Schema Design 15

3.2.3 Interface Design (UI/UX) 16

3.2.4 Physical DFD 18

3.2.5 System Flow Chart and Working Mechanism 18
3.3 Algorithm Details 20
CHAPTER 4: IMPLEMENTATION AND TESTING 22
4.1 Implementation 22
4.1.1 Tools Used (CASE Tools, Programming Language, Database Platforms) 22
4.1.2 Implementation Details of Modules (Description of procedures/ Functions/ Methods 23
4.2 Testing 29
4.2.1 Test Cases for Unit Testing 29
4.2.2. Test Cases for System Testing 32
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS 35
5.1. Conclusion 35
5.2 Lessons Learned/Outcome 35
5.3 Future Recommendations 35
REFERENCE
APPENDIX: SYSTEM SCREENSHOTS
SOURCE CODE

LIST OF ABBREVIATIONS

Abbreviation	Description
CMS	Canteen Management System
CSS	Cascading Style Sheet
DFD	Data Flow Diagram
HTML	Hypertext Markup Language
JS	JavaScript
KNN	K-Nearest Neighbors
SQLite	Structured Query Language (SQL) Lite

LIST OF FIGURES

Figure 3. 1 Use Case for CanteenMate 8
Figure 3. 2 Gantt Chart for CanteenMate 10
Figure 3. 3 ER Diagram for CanteenMate 11
Figure 3. 4 Context Diagram for CanteenMate 12
Figure 3. 5 Level 1 DFD for CanteenMate 13
Figure 3. 6 Three-tier architecture 14
Figure 3. 7 Database Schema Design for CanteenMate 15
Figure 3. 8 Login/Register Interface Design 16
Figure 3. 9 User Dashboard Interface Design 16
Figure 3. 10 Cart Interface Design 16
Figure 3. 11 Admin Order View Interface Design 17
Figure 3. 12 Admin Menu Interface Design 17
Figure 3. 13 Physical DFD for CanteenMate 18
Figure 3. 14 System Flowchart for CanteenMate 19

LIST OF TABLES

Table 4. 1 Test case for User Registration of CanteenMate 29
Table 4. 2 Test case for User Login of CanteenMate 30
Table 4. 3 Test case for search in CanteenMate 30
Table 4. 4 Test case for admin login in CanteenMate 31
Table 4. 5 Test Case for Forgot Password in CanteenMate 31
Table 4. 6 User Login/ Register Module of CanteenMate App 32
Table 4. 7 Test case for Managing the menu items of CanteenMate 33
Table 4. 8 Test case for Managing the orders of CanteenMate 34

CHAPTER 1: INTRODUCTION

1.1 Introduction

Canteens serve as essential food service providers in institutions, offices, and public places. They cater to a large number of people daily, offering meals and refreshments conveniently. However, managing a canteen efficiently is challenging task, especially when relying on manual operations. Issues such as slow order processing, inventory mismanagement, and lack of transparency in financial transactions create inefficiencies that impact customer satisfaction and business profitability.

CanteenMate is a simplified canteen management system (CMS) designed to digitize and streamline daily operations by focusing on real-time order placement and cashless transactions, while deliberately excluding stock monitoring and automated reporting features to keep the system lightweight and user-focused. This approach aims to enhance the customer experience by reducing wait times, minimizing errors in order fulfillment, and offering seamless payment options through a centralized digital platform.

The growing reliance on digital solutions across various industries emphasizes the need for canteens to adopt technology- driven management approaches. CanteenMate provides a centralized digital platform to optimize workflow, ensure better financial tracking, and promote a user-friendly experience for both customers and administrators. The implementation of this system will significantly reduce operational delays, enhance productivity, and improve overall service quality.

Without stock monitoring, CanteenMate handles food wastage in a simple hands-on way by letting staff and admins take charge. They use past sales info and watch what's happening each day to figure out how much food to make, trying to match it with people actually eat. The system makes it easy for the customers to place orders in real time, so staff can adjust how much they cook based on those orders instead of making big batches ahead of time, which reduces the extra food wastage.

CanteenMate provides a centralized digital platform that not only optimizes workflow and financial tracking but also promotes transparency, sustainability (through reduces paper usage), and accountability. By implementing this system, canteens can significantly reduce operational delays, enhance productivity, minimize food wastage, and improve overall service quality thereby aligning with modern expectations in institutional and corporate environments.

1.2 Problem Statement

The conventional canteen management process involves several inefficiencies that hamper smooth operations and customer satisfaction. The primary challenges faced by canteens include:

- Long Waiting Times: Customers experience delays due to slow manual order processing, leading to dissatisfaction and crowding, especially during peak hours.
- Order Inaccuracies: Paper-based order tracking is prone to errors, resulting in incorrect or misplaced orders.
- Inefficient Payment Handling: Manual cash transactions increase the risk of errors, theft and delays in payment processing.
- Lack of Data Analytics: Traditional record-keeping methods make it difficult to analyze customer preferences.
- No Pre-ordering Facility: Customers cannot place orders in advance, which leads to congestion at counters and increased waiting time. It also affects the kitchen staff's ability to prepare food in a timely manner.
- Limited Transparency and Communication: Customer are not updated about their order status, leading to repeated follow-ups and confusion.

These challenges highlight the urgent need for a smart, automated canteen management system that can streamline operations and enhance user experience.

1.3 Objective

The objective is to carry out this project with the quality requirements as mentioned below:

- To develop a smart canteen management system that streamlines digital food ordering, payments, and real-time tracking about order status while enhancing user experience and administrative efficiency.
- To implement a collaborative filtering algorithm for personalized food recommendation and integrate a feedback system to boost user engagement and service quality.

1.4 Scope and Limitations

1.4.1 Scope

The CanteenMate system will be used for institutional and corporate canteens to make efficient food service operations. This system can be accessed by all types of users, including customers and admin, with authenticated usernames and passwords to place orders and manage tasks. It offers a user-friendly and visually appealing interface to make

ordering and navigation easy for customers. The system can be accessed by every user with internet access via mobile and web platforms.

1.4.2 Limitations

While CanteenMate aims to enhance canteen operations and customer satisfaction, several limitations should be considered:

- Reliance on stable internet and compatibility with existing hardware and software.
- Language may be a barrier for those who do not understand English, as the system is primarily in English.

1.5 Development Methodology

The project utilizes the Agile methodology while developing the website. This project benefits from iterative development, continuous feedback, and adaptability to changing requirements, making Agile a suitable choice. Agile is a software development methodology that emphasizes flexibility, collaboration, and customer feedback. It breaks the project into small, manageable increments called sprints, allowing the team to deliver functional components regularly. Agile promotes adaptive planning, evolutionary development, early delivery, and continuous improvement throughout the development process.

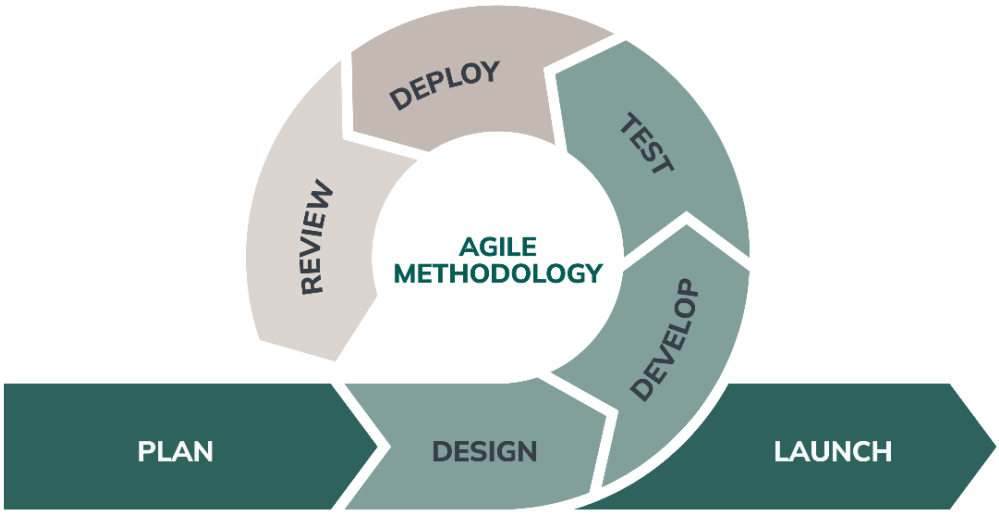


Figure 1. 1 Agile Methodology [1]

Following are the reasons behind using Agile methodology for development:

- Agile adapts to changing requirements throughout the project.
- Continuous feedback ensures the product meets user needs.
- Risks are addressed early through constant iterations.
- Regular communication helps identify and solve issues quickly.

1.6 Report Organization

Introduction

This chapter deals with the introduction of the CanteenMate system, including its objectives and limitations, along with the reasons for developing the system.

Background Study and Literature Review

This chapter summarizes the work conducted in the field of canteen management systems and describes the features of existing applications related to the CanteenMate project.

System Analysis and Design

This chapter focuses on the different requirements of the system, which include functional and non-functional requirements, feasibility analysis, Structured Approach (Data Modelling: DFD, Architectural Design, Database Schema Design, Interface Design, Physical DFD), and the implementation details of the collaborative filtering algorithm.

Implementation and Testing

This chapter emphasizes the tools used in the development of the CanteenMate system, the implementation details, and the results of tests performed.

Conclusion and Future Recommendation

This chapter highlights a brief summary of lessons learned the outcomes and conclusion of the entire CanteenMate project, and explain what has been accomplished and what further improvements could be made.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Traditional canteen management relies heavily on manual processes, which often lead to inefficiencies and inconvenience for both customers and staff. Customers face long waiting times, order inaccuracies, and a lack of pre-ordering options, while canteen staff struggle with manual order processing and payment handling. This system often requires customers to visit the canteen in person, and staff must manage paper- based records or assist with order fulfillment without automated support. CanteenMate addresses these challenges by providing a digital platform that simplifies food ordering, payment processing, and real-time order tracking, allowing customers to place orders conveniently and enabling staff to manage operations without the burden of manual tasks.

2.2 Literature Review

Several similar applications and systems have been developed or are in progress available on the web, app stores, or as third-party solutions, some of which are free while others require purchase. The following are some of the existing systems:

- SmartCanteen: This is a cloud-based canteen management system that integrates online food ordering, payment processing, and inventory tracking. It offers a user-friendly platform for customers to place orders digitally and helps administrators manage stock and transactions efficiently. However, it may face challenges with high initial setup costs and requires reliable internet connectivity. [2]
- eCanteen: An online platform designed to optimize food ordering in educational institutions and corporate offices. It supports cashless transactions, automated order tracking, and real-time reporting to improve efficiency. A disadvantage is its potential limitation in scalability for very large canteens dues to resource constraints. [3]
- FoodSwipe: A digital system that enables pre-ordering through a mobile app or website. It integrates with multiple payment gateways, provides real-time order status updates, and helps reduce food waste through better demand forecasting. However, it may lack advanced customization options for specific canteen needs. [4]
- CanteenPro: A cloud-based solution that automates food ordering, billing, and delivery tracking. It includes and intuitive dashboard for administrators to monitor sales, manage

menu pricing and analyze customer preferences. A drawback is its dependency on cloud services, which could lead to downtime if internet or server issues occur. [5]

- iCanteen: A mobile-first system created for educational environments, allowing students to order food in advance, assisting canteen staff with meal preparation, and supporting digital payments and analytics for administrators. Its limitation includes potential difficulty in integrating with older hardware systems. [6]

These systems highlight the growing trend toward digital solutions in canteen management, providing insights into features that CanteenMate can adopt or improve upon to meet user needs effectively. The identified disadvantages also suggest areas where CanteenMate can focus to offer a stronger solution.

CHAPTER: 3 SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

3.1.1 Requirement Analysis

i. Functional Requirement

Different functional requirements of the system have been identified and are listed as below:

For Customers:

- The system should allow customers to register.
- The system should allow customers to place orders online by selecting items, specifying quantities, and choosing pickup or delivery options.
- The system should allow customers to view order history and see the order status.

For Admin:

- The system should allow the admin to log in to the system.
- The system should allow the admin to manage orders, including updating order statuses.
- The system should allow the admin to add, edit, update, and delete menu items and categories.
- The system should allow the admin to read and respond to customer feedback.

USE CASE DIAGRAM

In CanteenMate, the use case diagram consists of customers who are allowed to register and log in. Customers can place orders, view their order history, and can see order status. They are also allowed to provide feedback and ratings. Admins can log in, manage orders and menu items, and handle customer feedback and requests, ensuring smooth operation of the system.

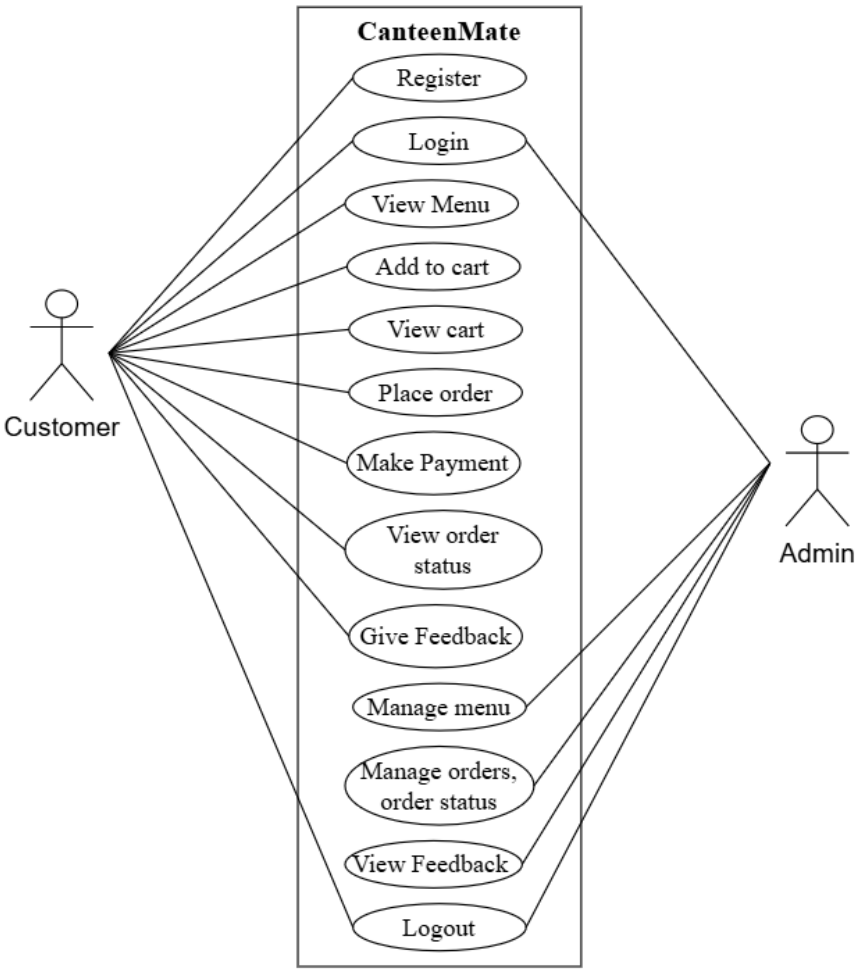


Figure 3. 1 Use Case for CanteenMate

ii. Non-Functional Requirement

Different non-functional requirements have been studied and identified and are listed as below:

- Security: The system is secure from outside attacks as only authorized users and customers are allowed to access the data. They can log into the system and have access to the CanteenMate web app, but access to various subsystems is protected by a user login screen requiring a username and password. This system uses passwords for security, along with different validation processes.
- Availability: The system will be running 24/7, so that canteen admin and customers can use it anytime.
- Performance: The system is designed to be quick and accurate. Its database is designed to handle orders and responds quickly to user actions.

- Reliability: It will work consistently ensuring accurate orders and data without a failure and has to be reliable due to importance of data and the damages that might happen due to incorrect or incomplete data.

3.1.2 Feasibility Analysis

• Technical Feasibility

The system is technically feasible as the requirement for the development of the system is easily accessible. The necessary hardware and software required for the development and implementation of the system is available. The basic programming language which is suitable for project is available and the libraries required for project is capable of achieving the result that we are aiming for. All the existing resources can be used for the development and maintenance system.

• Operational Feasibility

The system is easy to operate with the basic knowledge of computer and internet and well-trained manpower is not needed. User can easily access the system as it is user friendly in many aspects with good User Interface (UI). This system includes all the requirements used for CanteenMate and this system is completely operational and can be successfully implemented and administration feel easy to use this system as it is user-friendly.

• Economic Feasibility

The system is economically feasible and cost effective. As all the tools and resources required are either open sources or free. After the completion of the system, organization don't need to deploy any new hardware and software. The existing resource can be used.

• Schedule Feasibility Study

The system is complete within scheduled time and do not exceed the scheduled time.

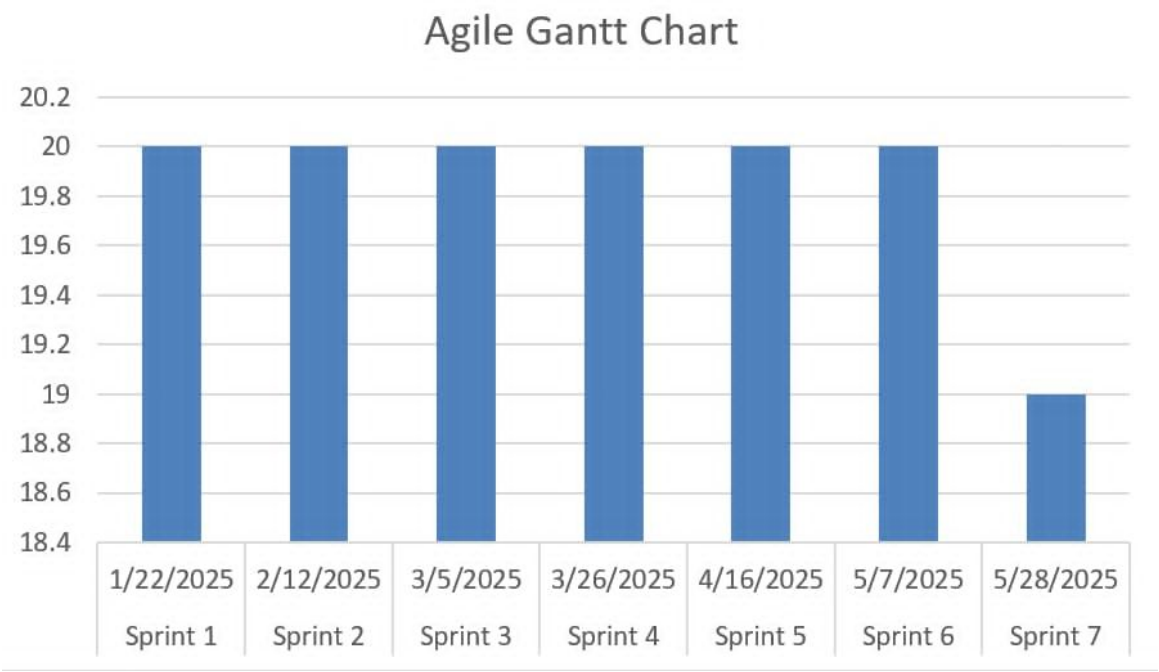


Figure 3. 2 Gantt Chart for CanteenMate

3.1.3 Data Modeling

This is an Entity-Relationship Diagram representing a canteen management system. It shows various entities such as user, order, order_item, order_addons, payments, items, categories, addons, addon_item, cart_items, menus, reviews. These entities are connected through relationships like has, contains, is in, is listed on, added to, includes, is part of, adds, places and is include in. Attributes for each entity are displays in ovals, such as name, id, description, price. The diagram illustrates the interactions between users, menus, items, orders, addons within the system.

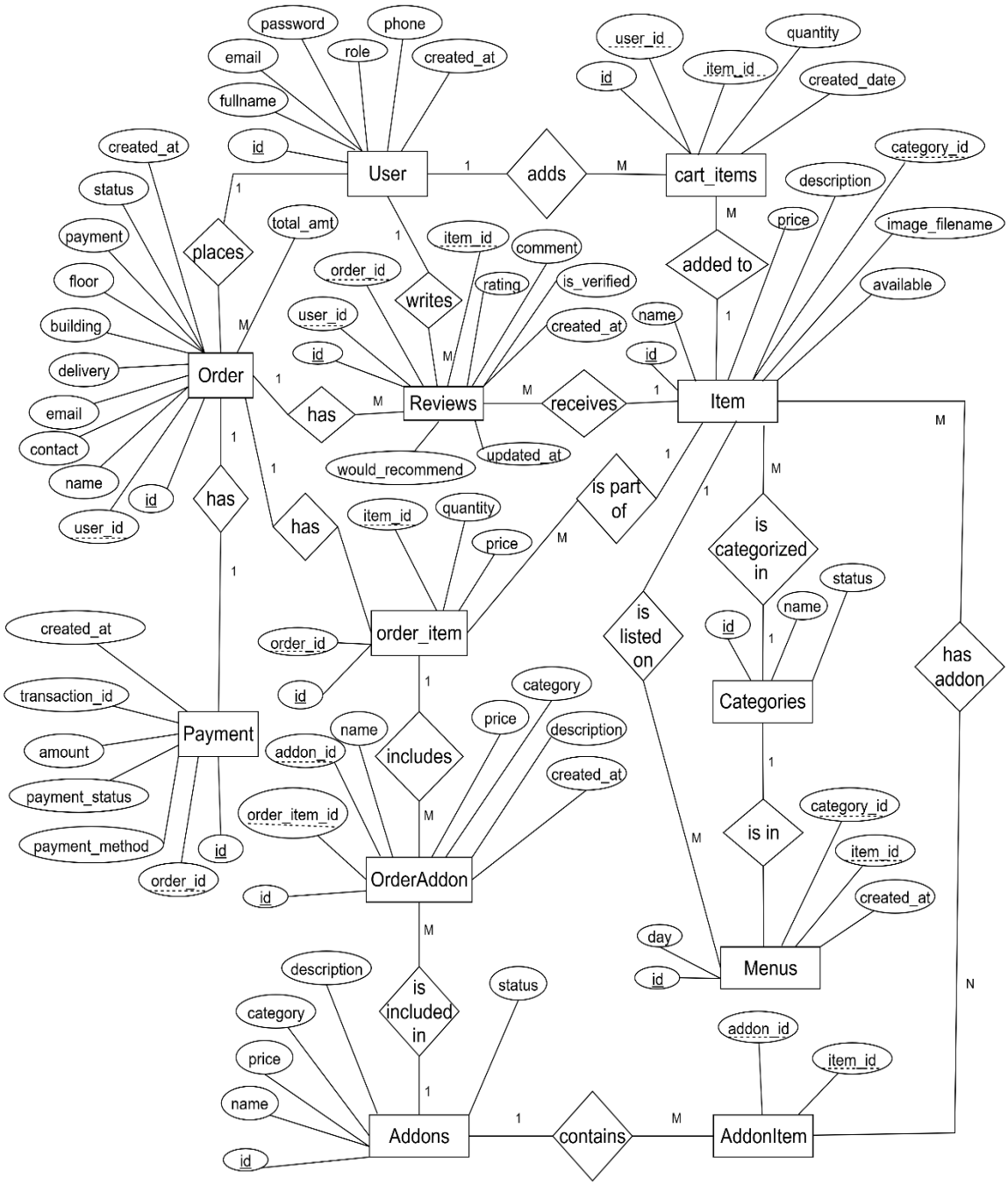


Figure 3. 3 ER Diagram for CanteenMate

3.1.4 Process Modeling: DFD

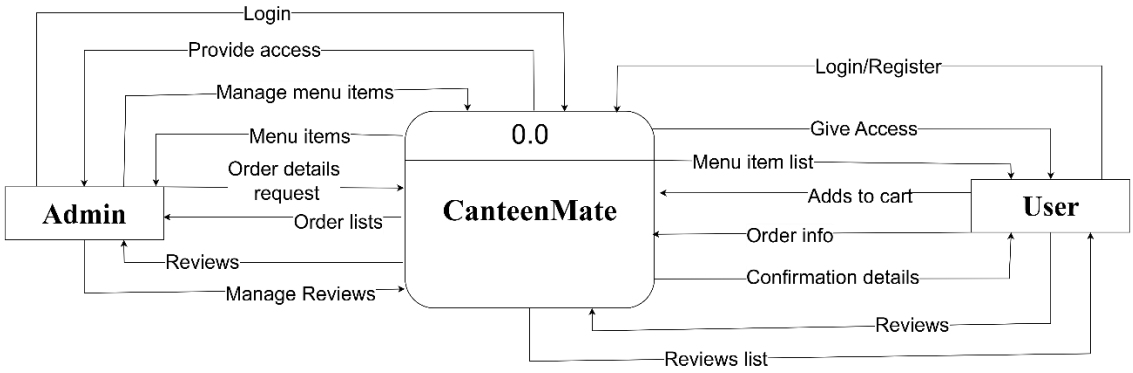


Figure 3. 4 Context Diagram for CanteenMate

The context diagram illustrates the interactions in the CanteenMate system, where the Admin and User communicate with the system to manage canteen food orders. The admin sends login requests, manages menu items, requests order details and lists, and manages reviews, receiving corresponding responses. The User interacts by sending login/ register requests, requesting menu item lists, adding items to the cart, requesting order information, and submitting reviews, receiving confirmation details, order information, and reviews, receiving confirmation details, order information, and reviews list accordingly. CanteenMate acts as a central hub, processing and facilitating these interactions.

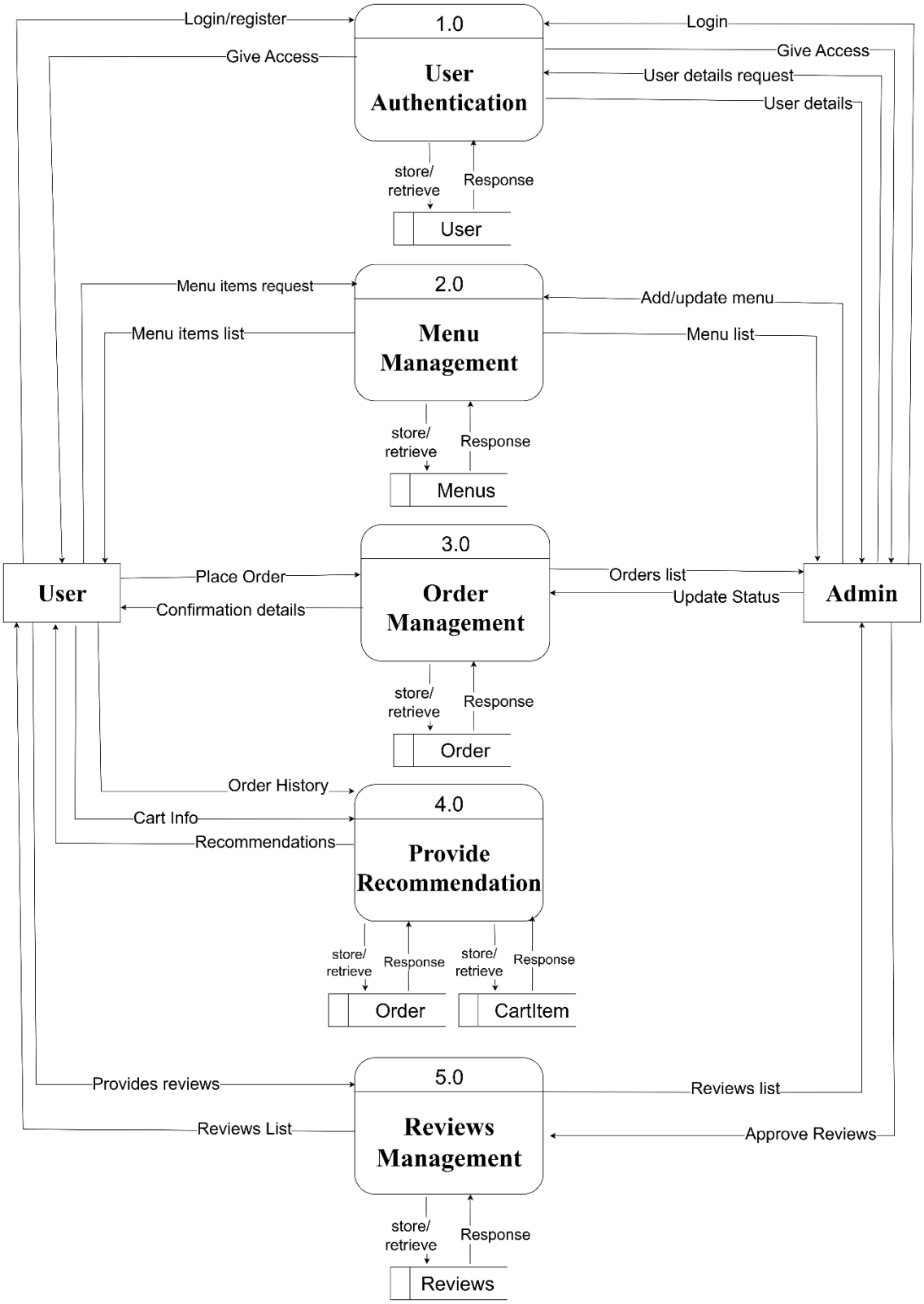


Figure 3. 5 Level 1 DFD for CanteenMate

This level 1 DFD shows the interactions in the CanteenMate system across five main modules: User Authentication, Menu Management, Order Management, Provide Recommendation, and Reviews Management. Users and Admin interact with the first

module for login/ register and user authentication. Users request menu items through second module, which also allows Admin to add/update menu items. The third module handles order management, where Users place orders and receive confirmation details, and Admin update order status. The fourth module provides recommendations and cart information to Users based on order history. Finally, the fifth module manages reviews, where Users provide reviews and receive reviews lists, and Admin approves reviews. Each module processes and stores relevant data, facilitating smooth interactions within the system.

3.2 System Design

To realize the different functional requirement of the system in graphical form, different design diagram of the system has been prepared which are as follows:

3.2.1 Architectural Design

The proposed system is a 3-Tier web-based system meaning a Client/Server Architecture in which the user interface, functional process logic, computer data storage, and data access are developed and maintained as independent modules, most often in different platforms.

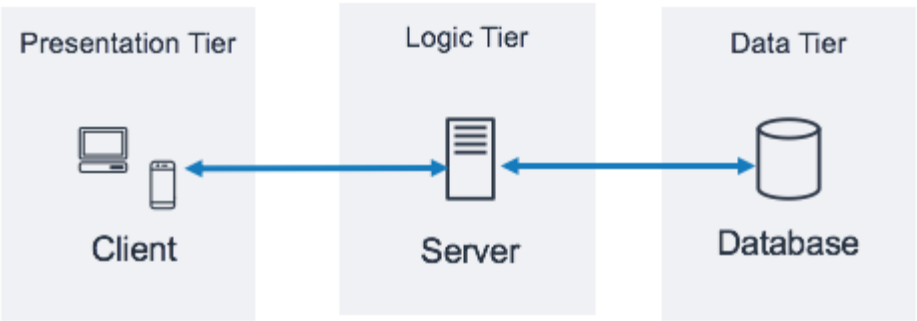


Figure 3. 6 Three-tier architecture [7]

1. **Client layer:** It is also called presentation layer. This is the part of CanteenMate that user sees and interacts. It's where customers place orders, view menus, or check their order status and admin manages orders and the reviews. This layer send user requests (like placing an order) to the next layer.
2. **Application Layer:** This is the brain of CanteenMate, handling all the works. It processes the requests from client layer, like checking if the order is valid or not, calculating the total price. It acts as a middleman, passing information between user interface and the database.
3. **Database Layer:** This is where all the data for CanteenMate is stored, like menu items, customer orders. When the application layer needs to save a new order this layer connects

to the database to make it happen. It keeps the data organized and separate from the other layers, so the system runs smoothly.

Following are the reasons behind choosing three tier architecture:

- It makes the logical separation among three different layers.
- It makes the maintenance process easier.
- It makes us easier to update on tier without involving other.

3.2.2 Database Schema Design

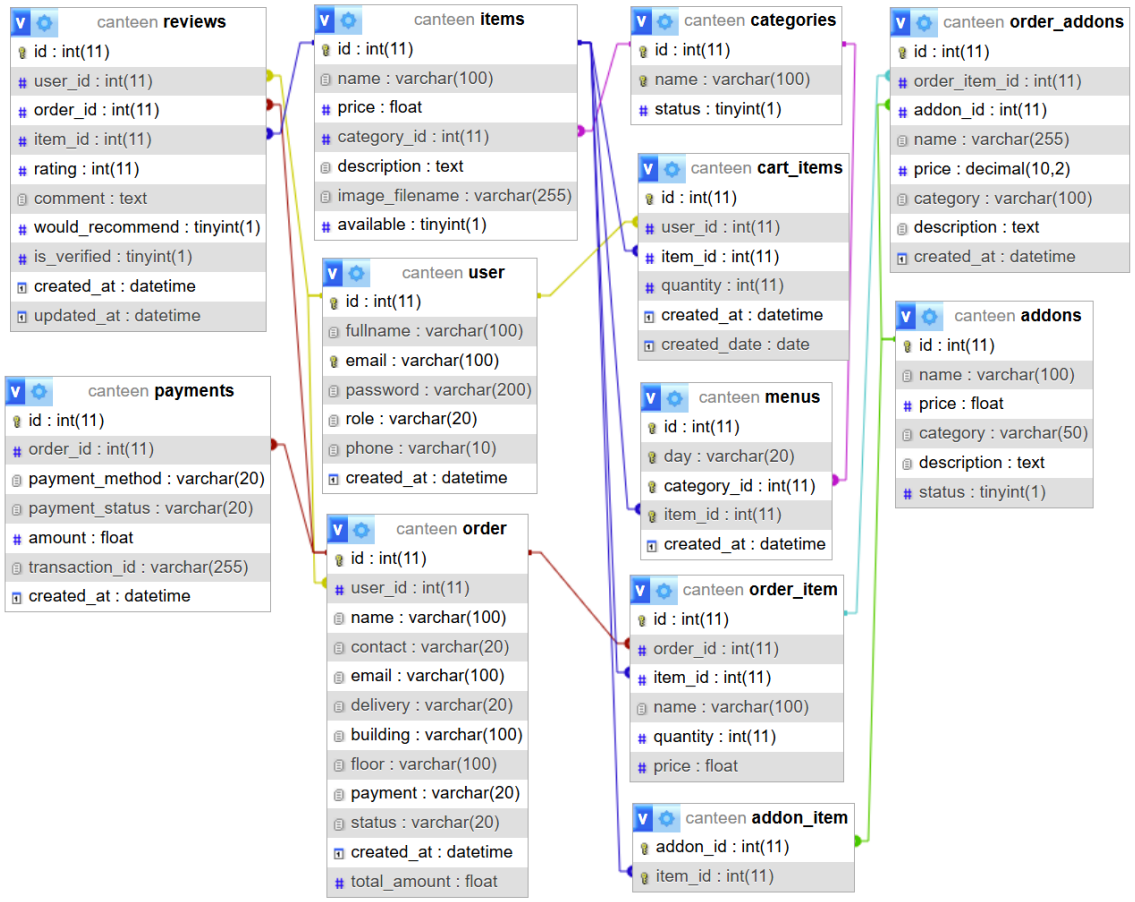


Figure 3. 7 Database Schema Design for CanteenMate

3.2.3 Interface Design (UI/UX)



Figure 3. 8 Login/Register Interface Design

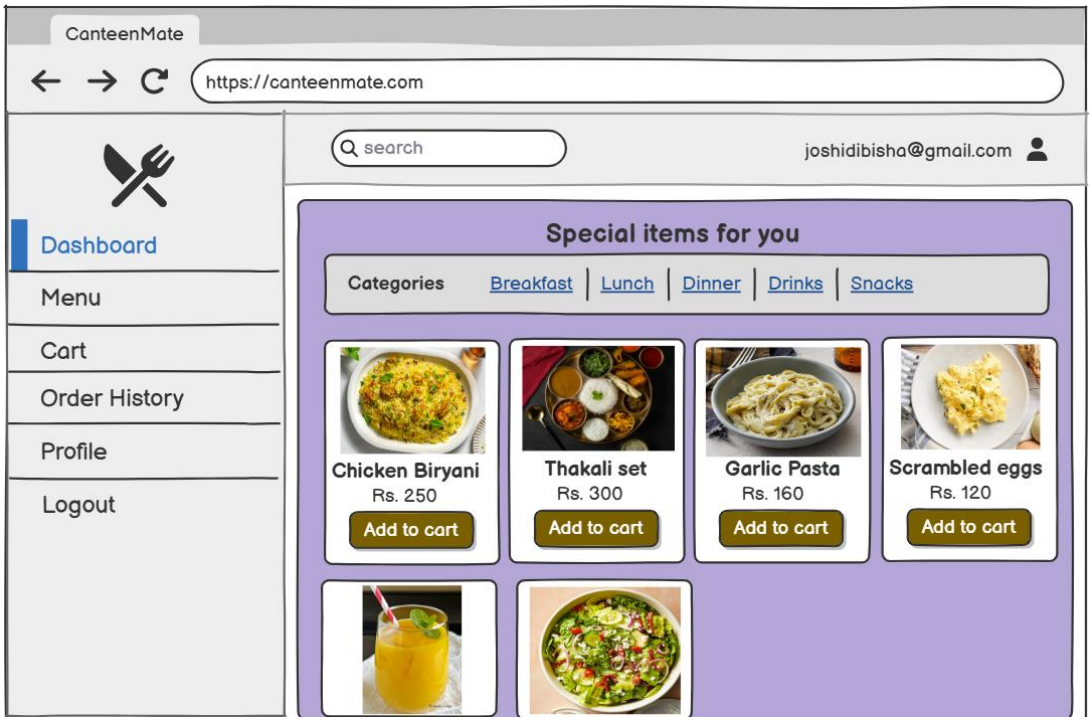


Figure 3. 9 User Dashboard Interface Design



Figure 3. 10 Cart Interface Design

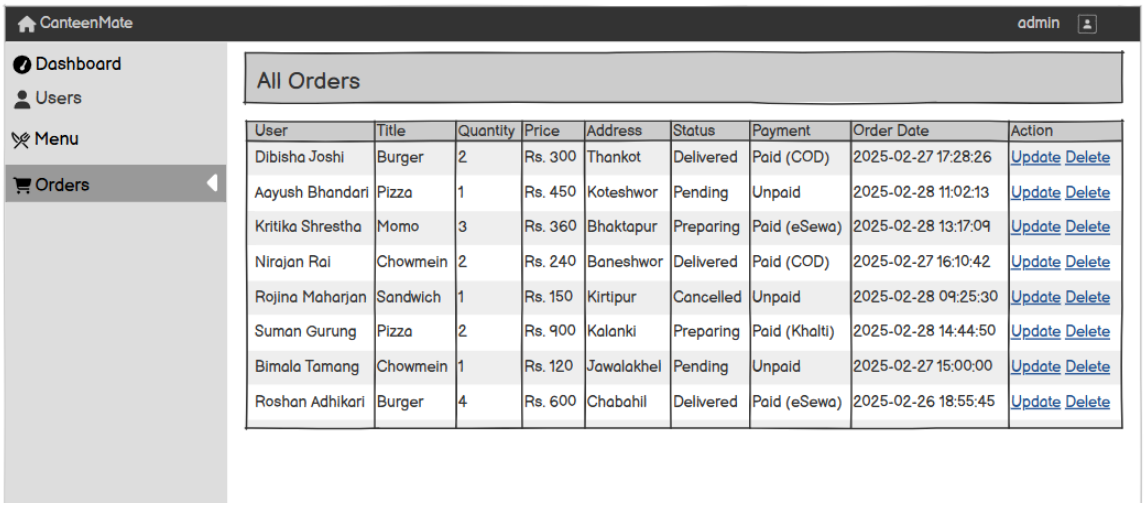


Figure 3. 11 Admin Order View Interface Design

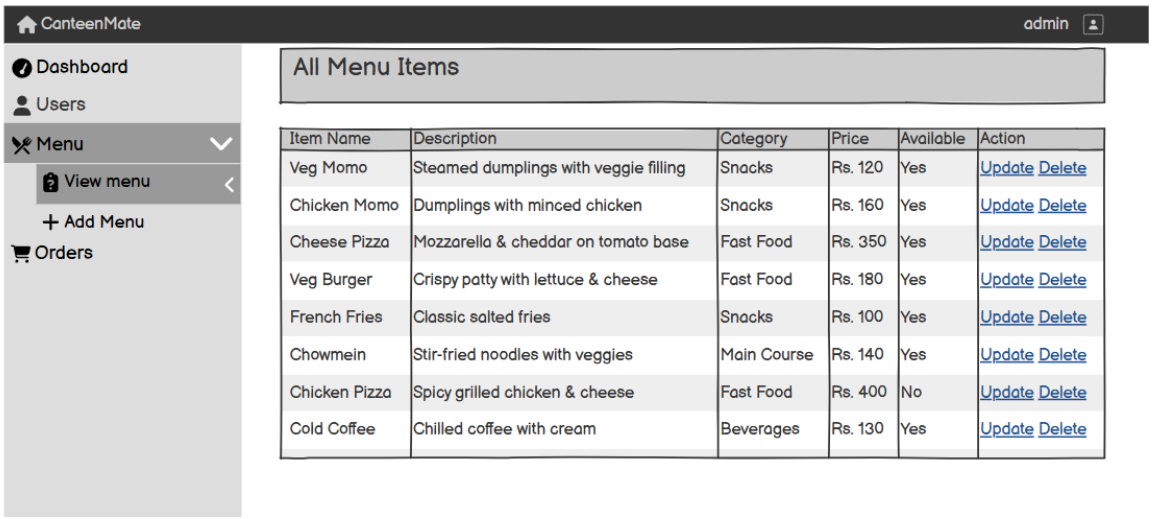


Figure 3. 12 Admin Menu Interface Design

3.2.4 Physical DFD

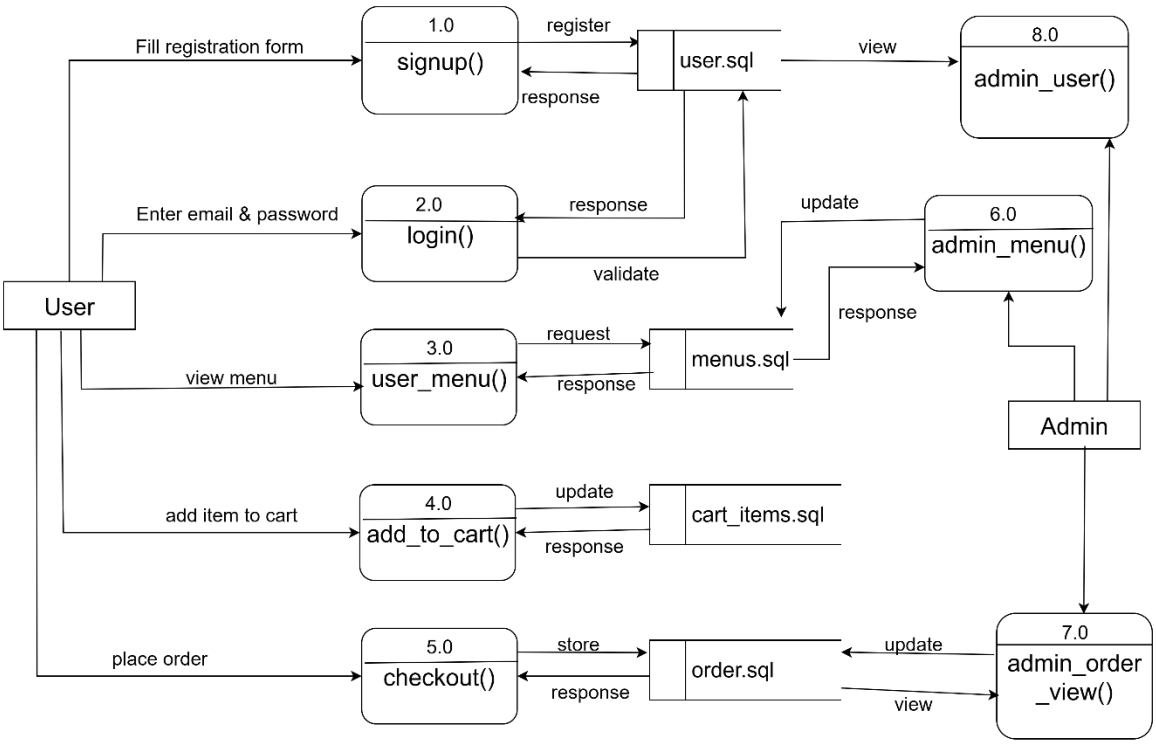


Figure 3. 13 Physical DFD for CanteenMate

3.2.5 System Flow Chart and Working Mechanism

The working mechanism of the system is explained below with the help of below system flow chart.

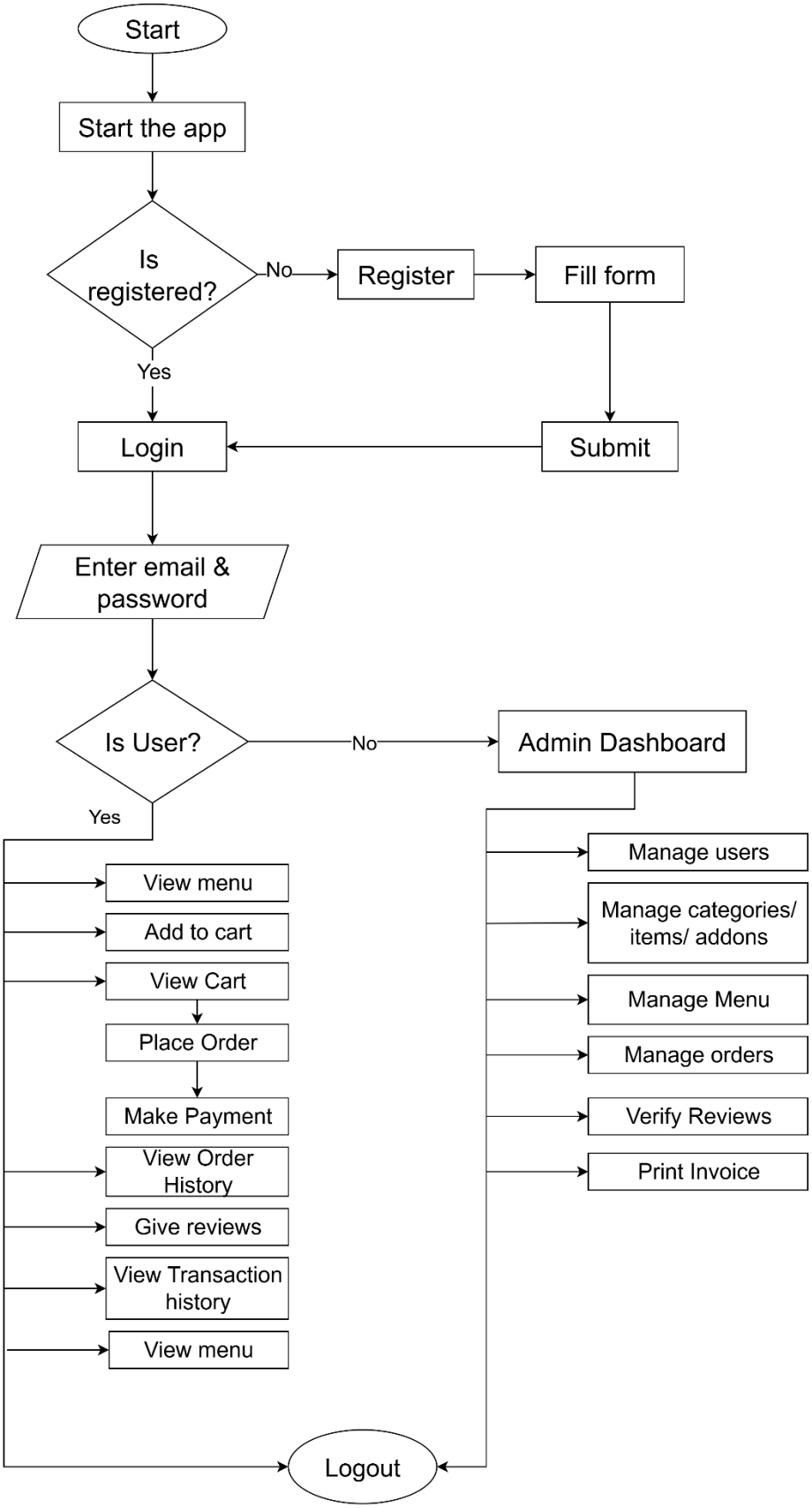


Figure 3. 14 System Flowchart for CanteenMate

3.3 Algorithm Details

This system uses a collaborative filtering algorithm based on a user-item matrix to recommend menu items to users. It uses the k-Nearest Neighbors(k-NN) algorithm with cosine similarity to identify users with similar ordering patterns and recommend items that these similar users have ordered but the target user has not. The system is made for the canteen of a organization, where recommendations are filtered to only include items available on the current day's menu (using the get_today_menu_item_ids function). Additionally, the system provides fallback recommendations based on item popularity (get_fallback_recommendations) and generates explanations (genereate_explanation) to enhance user trust and engagement.

Key Components

1. User-Item Matrix: The system constructs a matrix where rows represents users, columns represent menu items, and values indicate the quantity of items ordered by each user (build_user_item_matrix). This matrix captures user preferences based on their order history. [8]
2. K-NN Model: This system trains a k-NN model using cosine similarity to find users with similar ordering patterns (train_knn). The cosine similarity metric ensures that users with similar item preferences are grouped together, regardless of the absolute quantities ordered. [9]
3. Recommendation Logic: For a given user, the system identifies similar users, aggregates their orders, and recommends items the target user hasn't ordered, prioritizing those available on the current day's menu (recommend_items_for_user).
4. Fallback Mechanism: If collaborative filtering fails (e.g., no similar users or insufficient data), the system recommends popular items or new items from the menu (get_fallback_recommendations).

Additional Information Used:

- User data: User IDs and their order history (items and quantities ordered).
- Item data: Menu items, their availability on specific days (from the Menu table), and their order frequency (for fallback recommendations).
- Contextual data: The current day of the week to filter recommendations to only include items available on the menu.

The main idea of this collaborative filtering approach is to identify patterns in user behavior (i.e., which items users tend to order) and recommend items based on the preferences of similar users, ensuring relevance by aligning with the daily menu.

Similarity Distance Formula

This system uses cosine similarity to measure how similar two users' ordering patterns are, based on their vectors in the user-item matrix (where each vector represents quantities of items ordered). The cosine similarity formula is:

Cosine Similarity = $(A \cdot B) \ / \ (||A|| \times ||B||)$

This algorithm finds similarities to suggest relevant items, using order quantities. This system ensures recommendations filter the menu availability and provides explanations user-friendly, like "Recommended because similar user ordered this".

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used (CASE Tools, Programming Language, Database Platforms)

The following tools and frameworks were utilized for the development of CanteenMate project:

Front End Tools

- **HTML**

HTML is used to create the structure of the web pages, including forms, tables and page elements such as headers, footers, and the dashboard for the CanteenMate application.

- **CSS**

CSS is used to style the various components of the project, enhancing the visual appeal and user experience of the application.

- **JavaScript**

JavaScript is used to add interactivity and responsiveness to the CanteenMate project, improving user-friendliness and dynamic functionality.

- **Bootstrap**

Bootstrap is used to style HTML elements such as forms, tables, and tags like divs and spans, ensuring a consistent and responsive design.

Back End Tools

- **Python (Flask)**

Flask, a Python-based micro web framework, is used as the server-side technology to handle CRUD operations with the database and manage user sessions during interactions with the CanteenMate application.

Server

- **Flask Development Server**

The built-in Flask development server is used during development to run the application and test the integration of Python, Flask, and the database.

Database

- **SQLite**

SQLite is used as the database for the CanteenMate project to store and manage application data efficiently, leveraging its lightweight and serverless architecture.

Documentation Tools

- **MS Office**

MS Office is used for writing and editing the documentation of the CanteenMate project.

- **Draw.io**

This is used to generate diagrams for system analysis and design of CanteenMate project. Diagrams were created using this tool in order to save time since all components are available with drag and drop functions.

- **Balsamiq**

Balsamiq is used to design the user interface of the CanteenMate project.

4.1.2 Implementation Details of Modules (Description of procedures/ Functions/ Methods

The modules of the CanteenMate system are described as follows:

User Module

- **View Menu and Orders**

  In this module, users can browse and view the available menu items offered by the canteen. Users can also place orders for food items directly through the interface. The module retrieves menu data from the SQLite database and displays it using Flask templates.

- **User Order Placement**

  User can place orders by selecting items from the menu and can also select the addons and provide necessary details (e.g., quantity and delivery preferences). The module sends the order details to the server for processing and stores them in the database.

Login Module

In login module, user authentication is implemented. Users can log into the system using a valid email and password. The module supports both users and admins, with user roles defined in the SQLite database to differentiate between them. A “forgot password” feature allows users to reset their passwords. The module includes input validation and a “remember me” feature for session persistence. If login credentials are invalid, appropriate error messages are displayed.

Register Module

The register module enables users to create an account by providing their full name, email and password. It includes validations for required fields, email format password length, and password confirmation. The module checks for duplicate email addresses and assigns a default ‘user’ role to new accounts. Passwords are securely hashed before storage in the SQLite database. Upon successful registration, users are redirected to the login page.

Recommendation Module

The recommendation module provides personalized menu item suggestions to users based on their order history using a collaborative filtering approach with k-Nearest Neighbors (KNN). It builds a user-item matrix from past orders, trains a KNN model with a cosine similarity metric, and recommends items ordered by similar users that the current user hasn't ordered yet. If personalized recommendations are not possible (such as due to insufficient data), the module falls back to popular or new items.

Key Functions:

- **Build User-Item Matrix:** Creates a matrix of users and items based on order history, with quantities as values.

```
def build_user_item_matrix():
    orders = db.session.query(Order).all()
    data = []
    for order in orders:
        user_id = order.user_id
        for oi in order.items:
            data.append({
                'user_id': user_id,
                'item_id': oi.item_id,
                'quantity': oi.quantity
            })
    df = pd.DataFrame(data)
    if df.empty:
        return None, None
    user_item_matrix = pd.pivot_table(
        df,
        index='user_id',
        columns='item_id',
        values='quantity',
        aggfunc='sum',
        fill_value=0
    )
    return user_item_matrix, df
```

- **Train KNN Model:** Trains a KNN model on the user-item matrix using cosine similarity, adjusting the number of neighbors based on available data.

```
def train_knn(user_item_matrix, n_neighbors=3):
    if user_item_matrix is None or user_item_matrix.empty:
        return None, 0
    n_samples = user_item_matrix.shape[0]
    k = min(n_neighbors, n_samples - 1)
    if k <= 0:
        return None, 0
    model = NearestNeighbors(metric='cosine', algorithm='brute')
    model.fit(user_item_matrix.values)
    return model, k
```

- **Recommend Items with Explanations:** Generates personalized recommendations for a user by identifying similar users and recommending their ordered items, filtered by the current day's menu.

```
def recommend_items_for_user(user_id, user_item_matrix, model, n_neighbors=3, top_n=5):
    if user_item_matrix is None or user_id not in user_item_matrix.index or model is None:
        return []
    menu_item_ids = get_today_menu_item_ids()
    if not menu_item_ids:
        return []
    user_vector = user_item_matrix.loc[user_id].values.reshape(1, -1)
    n_samples = user_item_matrix.shape[0]
    k = min(n_neighbors, n_samples - 1)
    if k <= 0:
        return []
    distances, indices = model.kneighbors(user_vector, n_neighbors=k + 1)
    similar_user_indices = indices.flatten()[1:]
    similar_user_ids = user_item_matrix.index[similar_user_indices].tolist()
    if not similar_user_ids:
        return []
```

```
similar_users = User.query.filter(User.id.in_(similar_user_ids)).all()
similar_users_map = {}
for user in similar_users:
    if hasattr(user, 'name'):
        user_name = user.name
    elif hasattr(user, 'username'):
        user_name = user.username
    elif hasattr(user, 'email'):
        user_name = user.email.split('@')[0]
    elif hasattr(user, 'first_name') and hasattr(user, 'last_name'):
        user_name = f"{user.first_name} {user.last_name}"
    elif hasattr(user, 'first_name'):
        user_name = user.first_name
    else:
        user_name = f"User {user.id}"
    similar_users_map[user.id] = user_name
neighbors_matrix = user_item_matrix.loc[similar_user_ids]
summed_vector = neighbors_matrix.sum(axis=0)
user_items = user_item_matrix.loc[user_id]
unseen_items = summed_vector[user_items == 0]
if unseen_items.empty:
    return []
recommended_items = unseen_items.sort_values(ascending=False).head(top_n * 2)
recommendations = []
for item_id, popularity_score in recommended_items.items():
    if item_id not in menu_item_ids:
        continue
    item = Item.query.get(item_id)
    if not item:
        continue
    users_who_ordered = []
    for similar_user_id in similar_user_ids:
        if user_item_matrix.loc[similar_user_id, item_id] > 0:
            quantity = int(user_item_matrix.loc[similar_user_id, item_id])
```

```
            user_name = similar_users_map.get(similar_user_id, f"User {similar_user_id}")
            users_who_ordered.append({
                'name': user_name,
                'quantity': quantity
            })
        explanation = generate_explanation(users_who_ordered, item.name)
        recommendations.append({
            'item': item,
            'explanation': explanation,
            'explanation_type': 'collaborative',
            'popularity_score': int(popularity_score),
            'similar_users': users_who_ordered
        })
        if len(recommendations) >= top_n:
            break
    return recommendations
```

- **Fallback Recommendations:** Provides recommendations based on popular items or new items when personalized recommendations are not feasible, ensuring users always receive suggestions.

```
def get_fallback_recommendations (top_n=5):
    menu_item_ids = get_today_menu_item_ids()
    if not menu_item_ids:
        return []
    try:
        from sqlalchemy import func
        popular_items_query = db.session.query(
            Item,
            func.count(OrderItem.id).label('order_count')
        ).join(OrderItem, Item.id == OrderItem.item_id)\
         .group_by(Item.id)\
         .order_by(func.count(OrderItem.id).desc())\
         .limit(top_n * 2).all()
        recommendations = []
        for item, order_count in popular_items_query:
```

```
            if item.id not in menu_item_ids:
                continue
            explanation = f"Popular choice - ordered {order_count} times by other customers"
            recommendations.append({
                'item': item,
                'explanation': explanation,
                'explanation_type': 'popular',
                'order_count': order_count
            })
            if len(recommendations) >= top_n:
                return recommendations
    except Exception:
        pass
    fallback_items = Item.query.filter(Item.id.in_(menu_item_ids))\
                        .order_by(Item.id.desc())\
                        .limit(top_n).all()
    recommendations = []
    for item in fallback_items:
        recommendations.append({
            'item': item,
            'explanation': "New item - try something fresh!",
            'explanation_type': 'new'
        })
    return recommendations
```

Admin Module

- **Manage Menu**

Admins can add, edit, update, or delete menu items, including details like item name, price, description, categories (like “Breakfast, dinner, drinks”) and addons (like “Extra cheese”). This module interacts with the SQLite database to perform CRUD operations.

- **Manage Orders**

Admins can view, process, and update the status of user orders (e.g., pending, preparing, delivered).

- **Manage Reviews**

Admins can hide and verify the reviews given by the users.

4.2 Testing

System testing is done by giving different training and testing datasets. This test is done to evaluate whether the system is providing accurate summary or not. During the phase of the development of the system, our system is tested time and again. The series of testing conducted are as follow:

4.2.1 Test Cases for Unit Testing

In unit testing, we designed the entire system in modularized pattern and each module is tested until accurate results are achieved. Input forms are validated to prevent acceptance of invalid data.

User Registration

Table 4. 1 Test case for User Registration of CanteenMate

S. No	Test Name	Input	Expected Outcome	Actual Outcome	Test Result
1.	Enter invalid email, fullname, password and click signup button	Fullname: Dibisha Email:joshidibishagmail.com Password:TestingPass12 Confirm Password: TestingPass12	Please enter valid email	Registration failed	Pass
2.	Enter valid fullname, email, password and click signup button	Fullname: Dibisha Joshi Email:joshidibisha@gmail.com Password:dibi123 Confirm Password: dibi123	Account created successfully ! You can now log in.	Account created successfully, redirected to login	Pass

User Login

Table 4. 2 Test case for User Login of CanteenMate

S. No	Test Name	Input	Expected Outcome	Actual Outcome	Test Result
1.	Enter invalid email, password and click login button	Email:joshidibisha@gmail.com Password:Test@Pass	Please check your login details and try again.	Login failed	Pass
2.	Enter valid email, password and click login button	Email:joshidibisha@gmail.com Password:dibi123	Login successful, redirect to dashboard	Redirect to dashboard	Pass

Search Engine

Table 4. 3 Test case for search in CanteenMate

S. No	Test Name	Input	Expected Outcome	Actual Outcome	Test Result
1.	Enter invalid menu item name.	Keyword= ijhsdkn	No items found	No items available.	Pass
2.	Enter valid keyword.	Keyword= Yomari	Successful search result.	Seach results lists.	Pass

Admin Login

Table 4. 4 Test case for admin login in CanteenMate

S. No	Test Name	Input	Expected Outcome	Actual Outcome	Test Result
1.	Enter invalid email, password and click login button	Email:admingmail.com Password:admin 12	Please enter valid email	Invalid email or password	Pass
2.	Enter valid email, password and click login button	Email:admin@gmail.com Password:admin 123	Logged in successfully	Login successful redirected to dashboard	Pass

Forgot Password

Table 4. 5 Test Case for Forgot Password in CanteenMate

S. No	Test Name	Input	Expected Outcome	Actual Outcome	Test Result
1.	Enter invalid email and click reset button	Email:joshidibishagmail.com	Email not found	Email not found	Pass
2.	Enter valid email and click reset button	Email:joshidibisha@gmail.com	Email found	Email found, redirected to reset password page	Pass
3.	Enter invalid password and confirm password	Password:dibi123 Confirm Password: dibi@123	Password does not match.	Password does not match	Pass

4.	Enter valid password and confirm password and click reset button	Password:dibi123 Confirm Password: dibi123	Reset Password Successful	Reset Password Successful, redirect to login page	Pass

4.2.2. Test Cases for System Testing

In system testing, whole system is tested as below:

Table 4. 6 User Login/ Register Module of CanteenMate App

S. No.	Test Name	Input	Expected Outcome	Actual Outcome	Test Result
1.	Register a new user with valid full name, email, password and click register button.	Fullname= Dibisha Joshi Email: joshidbsa123@gmail.com Password=dibi123 Confirm Password: dibi123	Registration successful and redirected to the login page.	New user created and redirected to login page.	Pass
2.	Login to the system with newly created credentials.	Email: joshidbsa123@gmail.com Password: dibi123	Login Successful and redirect to dashboard.	Redirected to the dashboard.	Pass
3.	Logout from the system.	Click the logout button.	Logs out of the system.	User is logged out and redirected to login page.	Pass
4.	Forgot Password	Email = joshidbsa123@gmail.com Password: dibi@test	Password reset successful.	Password reset successful.	Pass

Manage Menu Items

Table 4. 7 Test case for Managing the menu items of CanteenMate

S. No.	Test Name	Input	Expected Outcome	Actual Outcome	Test Result
1.	Add a new menu item	Enter item name, price, description, choose image and select categories and click Save button.	Item added successfully and redirected to the item list page.	Item added successfully and redirected to the item list page.	Pass
2.	Update an existing menu item.	Enter new price of menu item 'Yomari' and click 'Save Item' button.	The item's price is updated, and the change is reflected in the item list.	Item was updated successfully and redirected to the item list page with changes reflected.	Pass
3.	Add a new item to the daily menu.	Select day 'Sunday' and select item in Newari Cuisine category, click 'Add to Menu' button.	Successfully added 1 item to Newari Cuisine menu for Sunday.	Successfully added item to menu for Sunday and redirected to the daily menu list page.	Pass
4.	Remove an existing daily menu item.	Navigate to Sunday's Newari Cuisine block, click 'Remove' for Yomari.	The item is removed from the daily menu for 'Sunday' and no longer appears in the list.	Item was removed and no longer visible in the daily menu list for 'Sunday'.	Pass

Manage Orders

Table 4. 8 Test case for Managing the orders of CanteenMate

S. No.	Test Name	Input	Expected Outcome	Actual Outcome	Test Result
1.	Place a new order by user (customer).	Add Item: Yomari to the cart, set Quantity:2, fill checkout details, and click 'Place Order'.	The order is placed successfully and redirected to the 'My orders' page.	Order was placed successfully, confirmation message displayed, and redirected to the order history page.	Pass
2.	Verify the list of orders is displayed correctly	Navigate to 'Orders' page observe order details table.	The order details are displayed correctly in the table.	The order details are displayed correctly in the table.	Pass
3.	Change order status by admin	Navigate to 'Orders' page, select order, change status to 'Delivered', click 'Update' button.	The order status is updated to 'Delivered' shown in the order list.	The order list status was updated and reflected in the order list.	Pass

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1. Conclusion

CanteenMate is a simple and user-friendly app designed to make canteen operations and food ordering easy for both customers and admins. Customers can browse daily menus, place orders, and check their order history, while admins can manage menu items, update daily menu, and handle order status. I have used HTML, CSS, JavaScript, Bootstrap, and Flask (Python web development framework) to develop this system. This system will run over internet. This system is reliable and straight-forward, making it easy for customer to order meal and receive it and for admins to keep the canteen operations organized and efficient.

5.2 Lessons Learned/Outcome

Every project makes us to learn and gain the knowledge in different aspects. In the following project, I have learned lots of problem-solving skills and learn things like finding the solution on my own, proper use of guidelines and writing skills.

- **Problem Solving Skills**

  From this project, I have learned lots of problem- solving skills and also learned to recognize different errors that occur in the system and solve it.

- **Writing Skills**

  I have learned how to prepare proposal and documentation related with project and also learned to use different case tools for use case diagram, data flow diagram, and ER-diagram and so on.

- **Manage Time**

  The most important lesson learnt was management of time according to the complexity of the system components i.e. know which components to prioritize.

5.3 Future Recommendations

In the future this system will be helpful for those users who are seeking a convenient and efficient way to order food and for canteen administrators looking to manage operations seamlessly. The future plans for this system are as follows:

- Remove bug from the system
- More security feature implementation

REFERENCE

[1] "Agile software development, nexapp," [Online]. Available: https://www.nexapp.ca/en/blog/agile-software-development. [Accessed Jan 2025].

[2] "SmartCanteen – A Cloud-Based Canteen Management System," [Online]. Available: https://app.smartcanteen.ae/.

[3] "eCanteen Digital Food Ordering System," [Online]. Available: https://www.ecanteen.com.au/.

[4] "FoodSwipe: Contactless Ordering and Payment System," [Online].

[5] "CanteenPro: Web-Based Canteen Management Solution," [Online]. Available: https://canteenpro.com/.

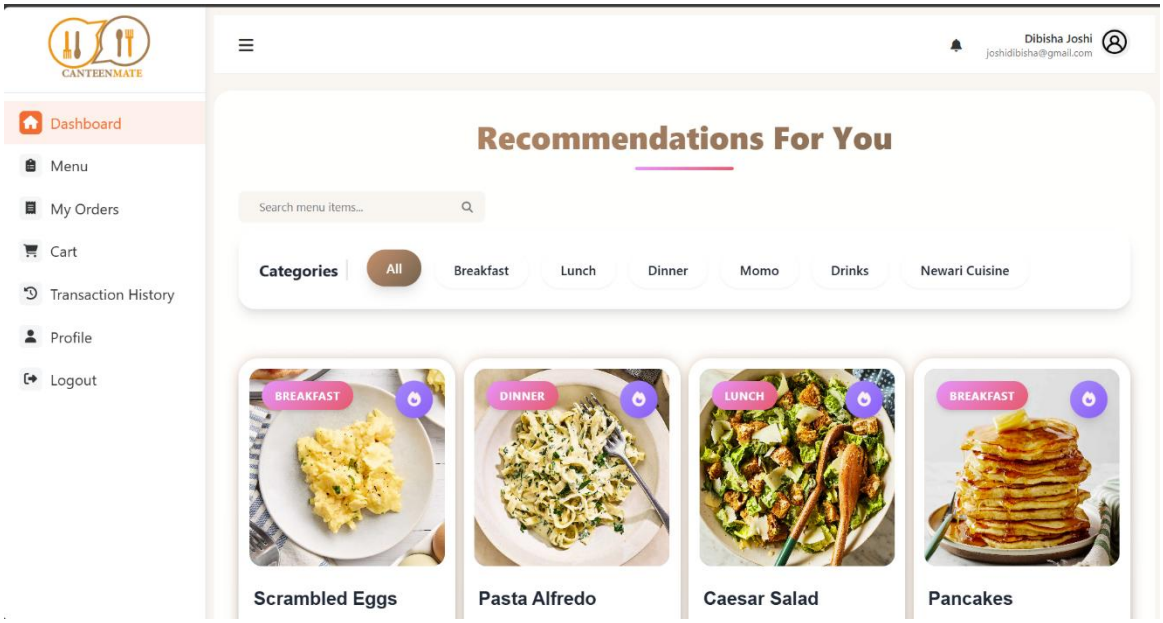
[6] "iCanteen: Canteen Management System," [Online]. Available: https://icanteen.co.uk/.

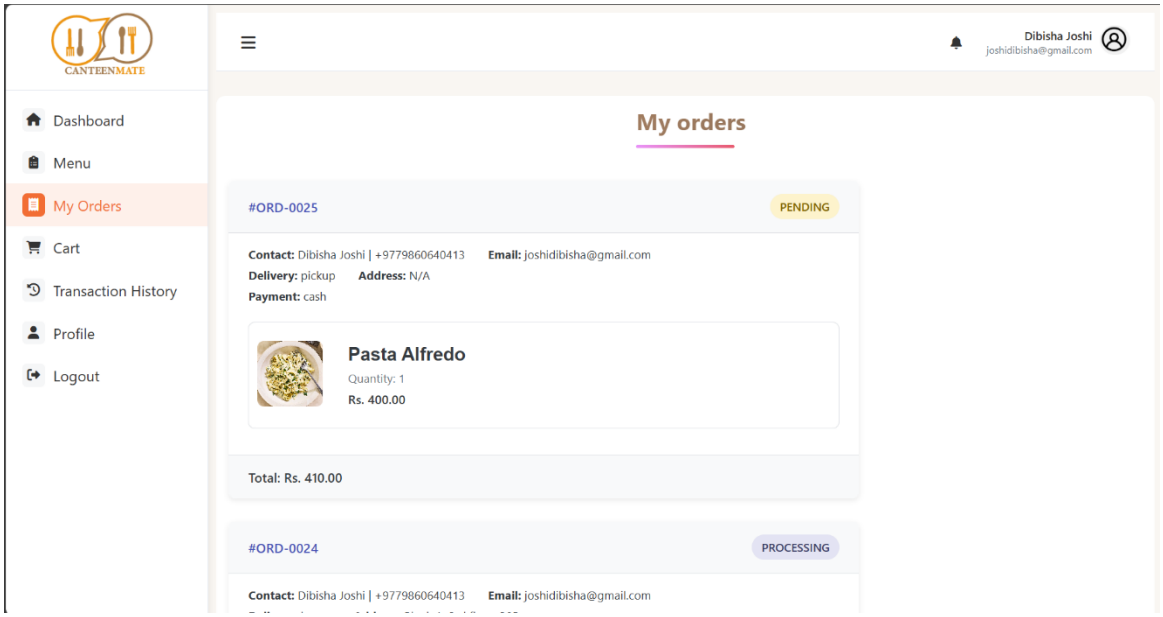
[7] "cloudbytes," [Online]. Available: https://cloudbytes.dev/aws-academy/creating-a-3-tier-network-architecture-vpc-with-aws-cdk-in-python.

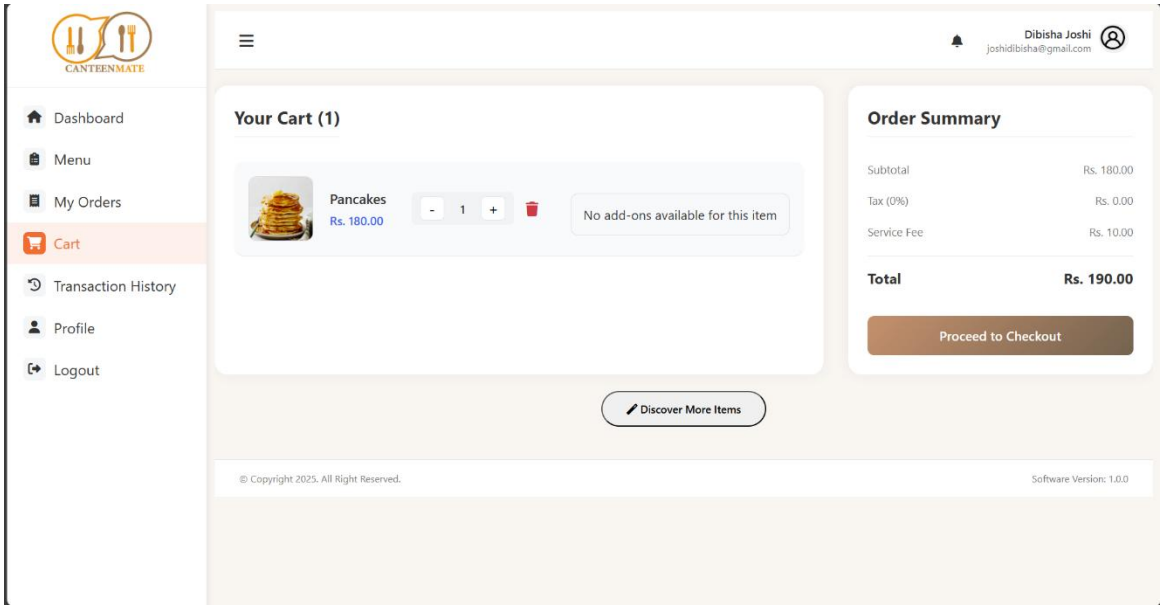
[8] N. B. M. e. al., "Collaborative filtering, K-nearest neighbor and cosine similarity in home decor recommender systems,arXiv preprint," 2024.

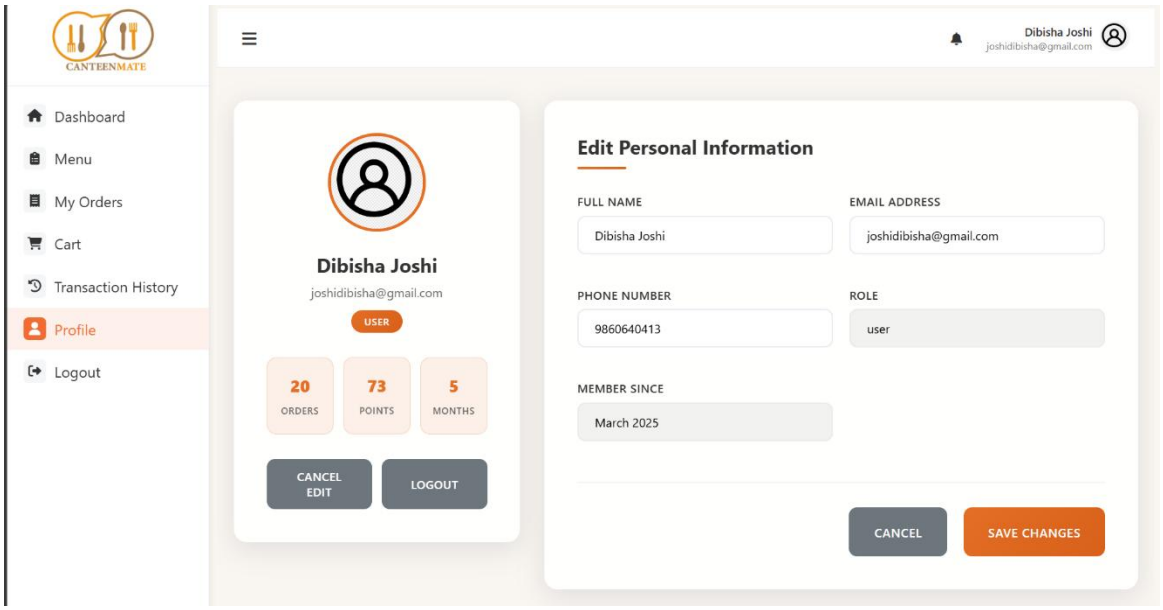
[9] "Nearest Neighbors — scikit-learn 1.6.0 Documentation," [Online].

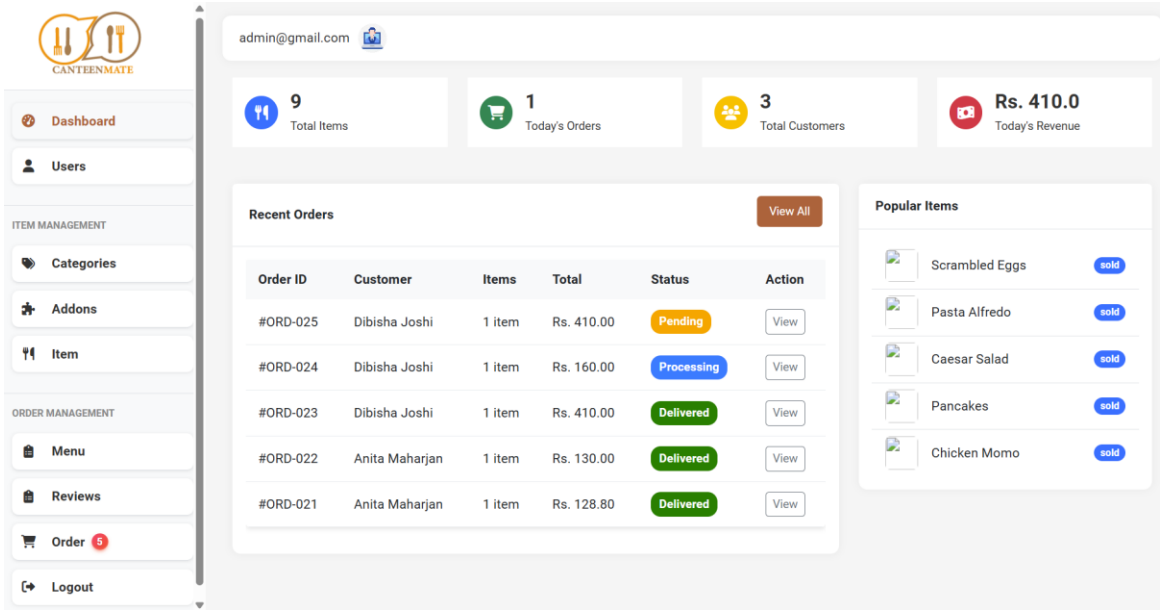
APPENDIX: SYSTEM SCREENSHOTS

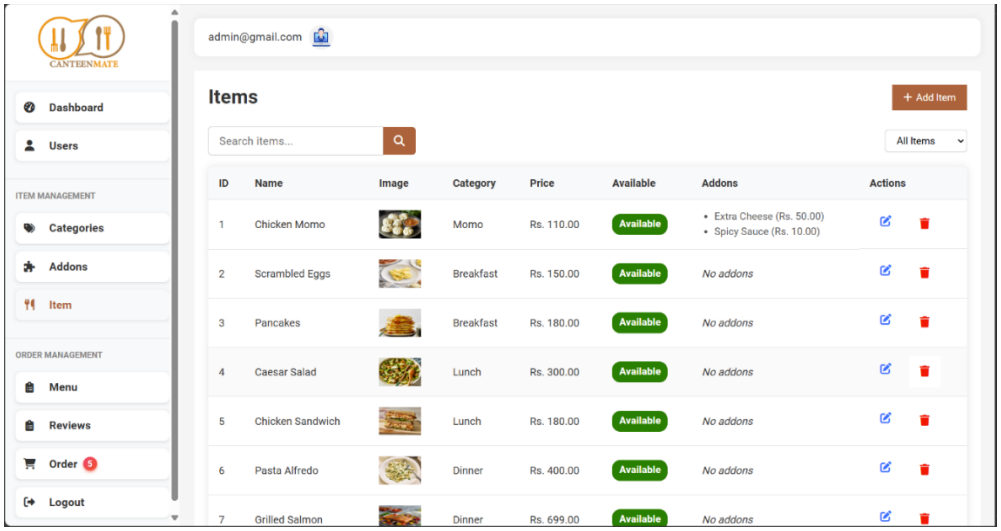


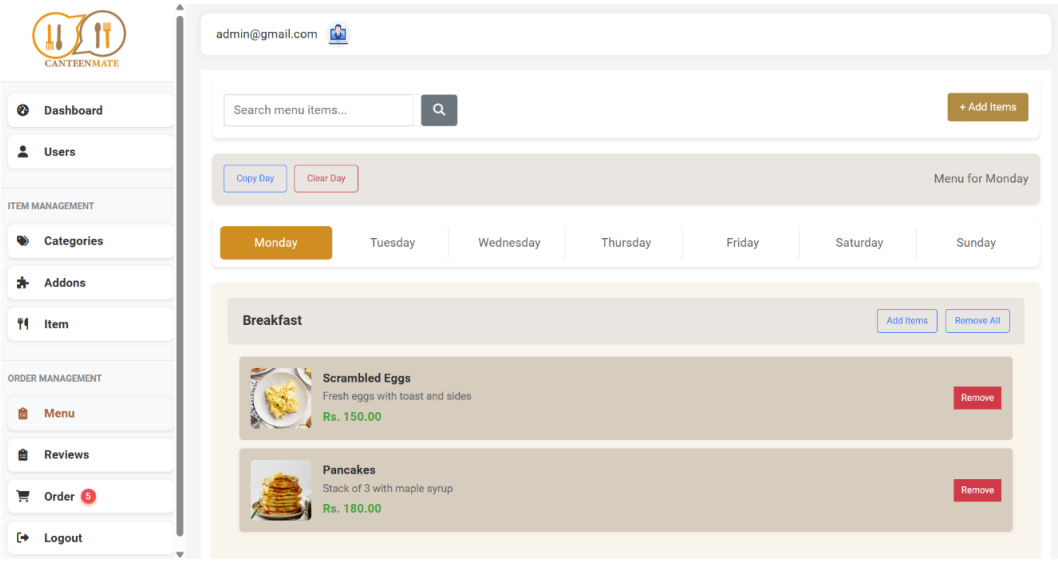


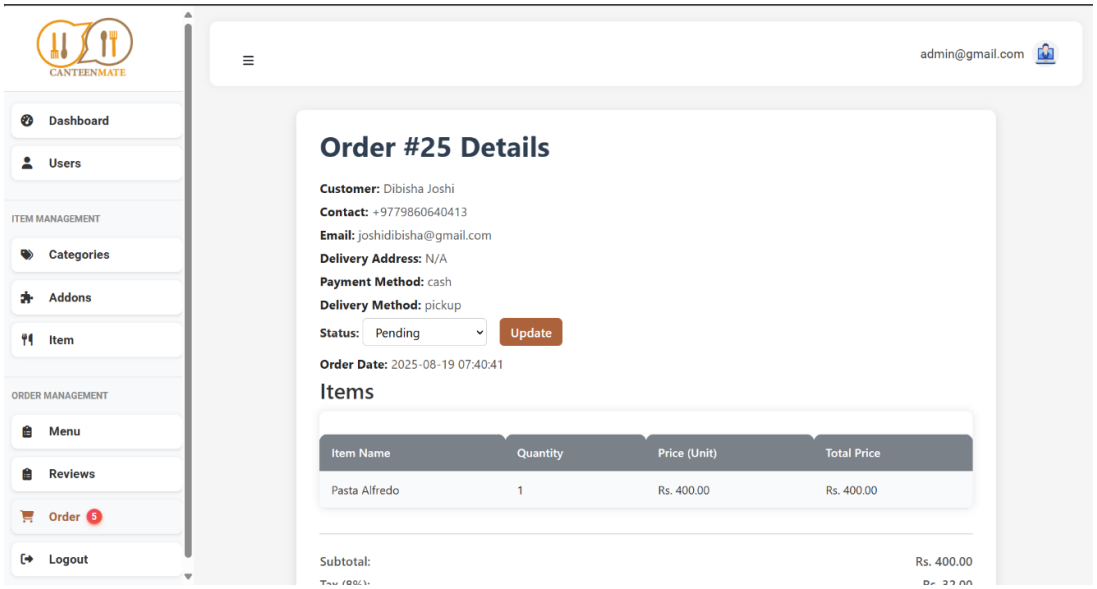

CANTEENMATE
Dashboard
Menu
My Orders
Cart
Transaction History
Profile
Logout
Dibisha Joshi
joshidibisha@gmail.com
Your Cart (1)
Pancakes
Rs. 180.00
No add-ons available for this item
Order Summary
Subtotal
Rs. 180.00
Tax (0%)
Rs. 0.00
Service Fee
Rs. 10.00
Total
Rs. 190.00
Proceed to Checkout
Discover More Items
© Copyright 2025. All Right Reserved.
Software Version: 1.0.0


CANTEENMATE
Dashboard
Menu
My Orders
Cart
Transaction History
Profile
Logout
Dibisha Joshi
joshidibisha@gmail.com
Dibisha Joshi
joshidibisha@gmail.com
USER
20
ORDERS
73
POINTS
5
MONTHS
CANCEL EDIT
LOGOUT
Edit Personal Information
FULL NAME
Dibisha Joshi
EMAIL ADDRESS
joshidibisha@gmail.com
PHONE NUMBER
9860640413
ROLE
user
MEMBER SINCE
March 2025
CANCEL
SAVE CHANGES


CANTEENMATE
admin@gmail.com
Dashboard
Users
ITEM MANAGEMENT
Categories
Addons
Item
ORDER MANAGEMENT
Menu
Reviews
Order 5
Logout
9
Total Items
1
Today's Orders
3
Total Customers
Rs. 410.0
Today's Revenue
Recent Orders
View All
Order ID
Customer
Items
Total
Status
Action
#ORD-025
Dibisha Joshi
1 item
Rs. 410.00
Pending
View
#ORD-024
Dibisha Joshi
1 item
Rs. 160.00
Processing
View
#ORD-023
Dibisha Joshi
1 item
Rs. 410.00
Delivered
View
#ORD-022
Anita Maharjan
1 item
Rs. 130.00
Delivered
View
#ORD-021
Anita Maharjan
1 item
Rs. 128.80
Delivered
View
Popular Items
Scrambled Eggs
sold
Pasta Alfredo
sold
Caesar Salad
sold
Pancakes
sold
Chicken Momo
sold


CANTEENMATE
admin@gmail.com
Dashboard
Users
ITEM MANAGEMENT
Categories
Addons
Item
ORDER MANAGEMENT
Menu
Reviews
Order 5
Logout
Items
+ Add Item
Search items...
All Items
ID Name Image Category Price Available Addons Actions
1 Chicken Momo Momo Rs. 110.00 Available • Extra Cheese (Rs. 50.00) • Spicy Sauce (Rs. 10.00)
2 Scrambled Eggs Breakfast Rs. 150.00 Available No addons
3 Pancakes Breakfast Rs. 180.00 Available No addons
4 Caesar Salad Lunch Rs. 300.00 Available No addons
5 Chicken Sandwich Lunch Rs. 180.00 Available No addons
6 Pasta Alfredo Dinner Rs. 400.00 Available No addons
7 Grilled Salmon Dinner Rs. 699.00 Available No addons


CANTEENMATE
admin@gmail.com
Dashboard
Users
ITEM MANAGEMENT
Categories
Addons
Item
ORDER MANAGEMENT
Menu
Reviews
Order 5
Logout
Search menu items...
+ Add Items
Copy Day
Clear Day
Menu for Monday
Monday Tuesday Wednesday Thursday Friday Saturday Sunday
Breakfast
Add Items
Remove All
Scrambled Eggs
Fresh eggs with toast and sides
Rs. 150.00
Remove
Pancakes
Stack of 3 with maple syrup
Rs. 180.00
Remove


CANTEENMATE
admin@gmail.com
Dashboard
Users
ITEM MANAGEMENT
Categories
Addons
Item
ORDER MANAGEMENT
Menu
Reviews
Order 5
Logout
Order #25 Details
Customer: Dibisha Joshi
Contact: +9779860640413
Email: joshidibisha@gmail.com
Delivery Address: N/A
Payment Method: cash
Delivery Method: pickup
Status: Pending
Update
Order Date: 2025-08-19 07:40:41
Items
Item Name Quantity Price (Unit) Total Price
Pasta Alfredo 1 Rs. 400.00 Rs. 400.00
Subtotal: Rs. 400.00

SOURCE CODE

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    if 'user_id' in session:
        return redirect(url_for('dashboard'))

    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')
        remember = True if request.form.get('remember') else False

        if not email or not password:
            flash('Email and password are required.', 'warning')
            return redirect(url_for('login'))
        if '@' not in email or '.' not in email:
            flash("Enter a valid Email address.",'warning')

        if len(password) < 6:
            flash('Password must be at least 6 characters long.', 'warning')
            return redirect(url_for('login'))

        user = User.query.filter_by(email=email).first()

        if not user or not check_password_hash(user.password, password):
            flash('Please check your login details and try again.', 'danger')
            return redirect(url_for('login'))

        session['user_id'] = user.id
        session['user_email'] = user.email
        session['user_role'] = user.role
        session['user_fullname'] = user.fullname

        if remember:
            session.permanent = True

        flash('Logged in successfully!', 'success')
        return redirect(url_for('dashboard'))
```

```python
@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        fullname = request.form.get('fullname')
        email = request.form.get('email')
        password = request.form.get('password')
        confirm_password = request.form.get('confirm_password')

        if not fullname or not email or not password or not confirm_password:
            flash('All fields are required.', 'warning')
            return redirect(url_for('signup'))

        if '@' not in email and '.' not in email:
            flash("Enter a valid Email address.",'warning')
            return redirect(url_for('signup'))
        if len(password) < 6:
            flash('Password must be at least 6 characters long.', 'warning')
            return redirect(url_for('signup'))

        if password != confirm_password:
            flash('Passwords do not match!', 'danger')
            return redirect(url_for('signup'))

        existing_user = User.query.filter_by(email=email).first()
        if existing_user:
            flash('Email address already exists.', 'danger')
            return redirect(url_for('signup'))

        new_user = User(
            fullname=fullname,
            email=email,
            role='user',
            password=generate_password_hash(password, method='pbkdf2:sha256')
        )
```

```python
@app.route('/admin/dashboard')
@login_required
def dashboard():
    role = session.get('user_role')
    user_id = session.get('user_id')
    if role == 'admin':
        user = User.query.get(user_id)
        total_items = Item.query.count()
        total_customers = User.query.filter(User.role != 'admin').count()
        today = date.today()

        start_datetime = datetime.combine(today, time.min)
        end_datetime = datetime.combine(today, time.max)

        todays_orders = Order.query.filter(
            Order.created_at >= start_datetime,
            Order.created_at <= end_datetime
        ).all()

        todays_orders_count = len(todays_orders)

        todays_revenue = sum(order.total_amount for order in todays_orders)

        recent_orders = Order.query.order_by(Order.created_at.desc()).limit(5).all()

        popular_items = db.session.query(
            OrderItem.item_id,
            OrderItem.name,
            db.func.count(OrderItem.item_id).label('count')
        ).group_by(OrderItem.item_id).order_by(db.desc('count')).limit(5).all()

        order_status_counts = db.session.query(
            Order.status, db.func.count(Order.id)
        ).group_by(Order.status).all()

        return render_template('admin/admin_dashboard.html',
```

```python
@app.route('/admin/categories')
@login_required
def categories():
    categories = Category.query.all()
    user = User.query.get(session['user_id'])
    return render_template('admin/categories.html', categories=categories, user=user, active_page='categories')
@app.route('/admin/categories/add', methods=['POST'])
@login_required
def add_category():
    name = request.form.get('name')
    status_str = request.form.get('status', 'inactive')
    status = True if status_str == 'active' else False

    if not name:
        flash('Category name is required.', 'danger')
        return redirect(url_for('categories'))

    new_category = Category(name=name, status=status)
    db.session.add(new_category)
    db.session.commit()

    flash('Category added successfully!', 'success')
    return redirect(url_for('categories'))

@app.route('/admin/categories/edit/<int:id>', methods=['POST'])
def edit_category(id):
    category = Category.query.get_or_404(id)
    category.name = request.form['name']
    category.status = request.form['status'] == 'active'
    db.session.commit()

    return redirect(url_for('categories'))
```

```
    {% block extra_css %}{% endblock%}
</head>
<body>
    <div class="container">
        <!-- Sidebar -->
        <div class="sidebar">
            <div class="logo">
                <img src="{{ url_for('static', filename='img/logo.png') }}" alt="Canteen Logo">
            </div>

            <div class="sidebar-menu">
                <a href="{{ url_for('user_dashboard') }}" class="side-menu-item {% if active_page == 'dashboard' %}active{% endif %
                    <div class="menu-icon"><i class="fas fa-home"></i></div>
                    <div class="menu-text">Dashboard</div>
                </a>

                <a href="{{ url_for('user_menu')}}" class="side-menu-item {% if active_page == 'menu' %}active{% endif %}">
                    <div class="menu-icon"><i class="fas fa-clipboard-list"></i></div>
                    <div class="menu-text">Menu</div>
                </a>

                <a href="{{ url_for('orders') }}" class="side-menu-item {% if active_page == 'orders' %}active{% endif %}">
                    <div class="menu-icon"><i class="fas fa-receipt"></i></div>
                    <div class="menu-text">My Orders</div>
                </a>
                <a href="{{ url_for('cart')}}" class="side-menu-item {% if active_page == 'cart' %}active{% endif %}">
                    <div class="menu-icon"><i class="fas fa-shopping-cart"></i></div>
                    <div class="menu-text">Cart</div>
                </a>

                <a href="{{ url_for('transaction')}}" class="side-menu-item {% if active_page == 'transaction' %}active{% endif %}"
                    <div class="menu-icon"><i class="fas fa-history"></i></div>
                    <div class="menu-text">Transaction History</div>
                </a>

                <a href="{{ url_for('profile')}}" class="side-menu-item {% if active_page == 'profile' %}active{% endif %}">
                    <div class="menu-icon"><i class="fas fa-user"></i></div>
```

```
<div class="content">
  <h1 class="page-title">My orders</h1>
  {% with messages = get_flashed_messages(with_categories=true) %}
                    {% for category, message in messages %}
                        {% if 'order_' in category %}
                            <div class="alert alert-{{ category.split('_')[1] }} d-flex justify-content-between align-items-center
                                <span>{{ message }}</span>
                                <button type="button" class="btn-close ms-3" data-bs-dismiss="alert" aria-label="Close"></button>
                            </div>
                        {% endif %}
                    {% endfor %}
                {% endwith %}
  <div id="orders-wrapper">
    {% if orders %} {% for order in orders %}
    <div class="order-card">
      <div class="order-header">
        <div class="order-number">#ORD-{{ '%04d'|format(order.id) }}</div>
        <div class="order-status {{ order.status }}">{{ order.status }}</div>
      </div>

      <div class="order-body">
        <div class="order-info">
          <span
            ><strong>Contact:</strong> {{ order.name }} | {{ order.contact
            }}</span
          >
          <span><strong>Email:</strong> {{ order.email }}</span><br />
          <span><strong>Delivery:</strong> {{ order.delivery }}</span>
          <span
            ><strong>Address:</strong> {{ order.delivery_address or 'N/A'
            }}</span
          ><br />
          <span><strong>Payment:</strong> {{ order.payment }}</span>
        </div>
```

```
<div class="order-item">
  <div class="items-image">
    <img
      src="{{ url_for('static', filename='uploads/items/' ~ item.item.image_filename) if item.item and item.item.image_f
      alt="{{ item.name }}"
    />
  </div>
  <div class="item-details">
    <div class="item-name">{{ item.name }}</div>
    <div class="item-quantity">Quantity: {{ item.quantity }}</div>
    <div class="item-price">Rs. {{ '%.2f'|format(item.price) }}</div>

    {% if item.order_addons %}
    <div class="item-addons mt-2">
      <small><strong>Add-ons:</strong></small>
      <ul class="pl-3">
        {% for addon in item.order_addons %}
        <li>
          {{ addon.name }} (+Rs. {{ '%.2f'|format(addon.price) }})
        </li>
        {% endfor %}
      </ul>
    </div>
    {% endif %}
  </div>
</div>
{% endfor %}
</div>
</div>

<div class="order-footer">
  <div class="order-total">
```